

Научно-технологический университет «Сириус»

ОТЧЁТ

по домашнему заданию № 4

по дисциплине «Распределённые базы данных»

Репликация, распределение данных и FDW
Выполнение на VM11 + VM2 (пошагово, со скринами)

Выполнил:
студент группы M09-КИИ26
Ермаков Лаврентий

VM11 192.168.122.61 · VM2 192.168.122.60 · SSH db1/1 · PG db1_user/1

Сириус, 2026

1. Постановка и роли VM

Задание 4 по дисциплине «Распределённые БД» состоит из трёх частей: зеркало (streaming replication), разрез данных на две БД и доступ к внешней БД через postgres_fdw. Чтобы не ломать сдачу лаб 1-3 на исходной VM1 (192.168.122.59), в качестве учебного primary использована отдельная VM VM11 (ubuntu24.04-11 / db-server-11 / .61) — полная копия БД «как после 1 лабы». VM2 (.60) сначала зеркало, затем хранилище личного контура.

Роль	VM	IP / hostname
Primary задания 4 (university)	VM11 · ubuntu24.04-11	192.168.122.61 · db-server-11
Зеркало → personal	VM2 · ubuntu24.04-2	192.168.122.60 · db-server-2
Сдача лаб 1-3 (не трогали)	VM1 · ubuntu24.04	192.168.122.59 · db-server

Три части по ТЗ

- Часть 1: снимок полной БД + streaming-зеркало; скрипты лабы 1 читают обе стороны (на standby — только SELECT).
- Часть 2: откат к снимку; university на VM11 без ФИО; personal на VM2; Python с двумя подключениями.
- Часть 3: на VM11 настроить FDW к VM2; снова работать «как с одной БД» с хоста .61.



Рисунок U0 — временная шкала трёх частей задания 4

2. Подготовка: university_db на VM11 «как после лабы 1»

Перед частью 1 на VM11 поднята полная схема лабы 1 (направления, группы, студенты с ФИО, телефоны, предметы, оценки, пары, посещаемость). Это нужно, чтобы снимок и зеркало содержали тот же объём данных, что демонстрировался в задании 1 (≈240 студентов, ≈320 телефонов, 678 оценок и т.д.). Ниже — улучшенная ER-диаграмма и реальные скрины с VM11.

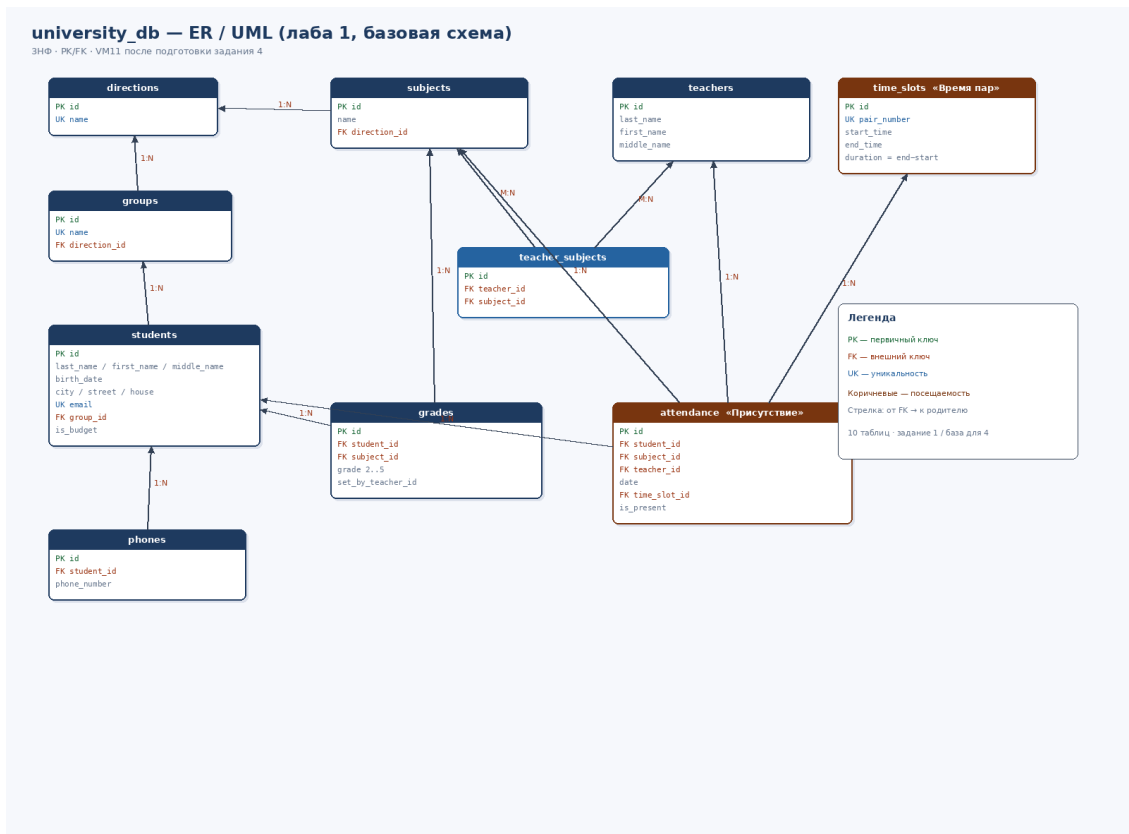


Рисунок 1 — улучшенная ER/UML university_db (лаба 1, база для задания 4)

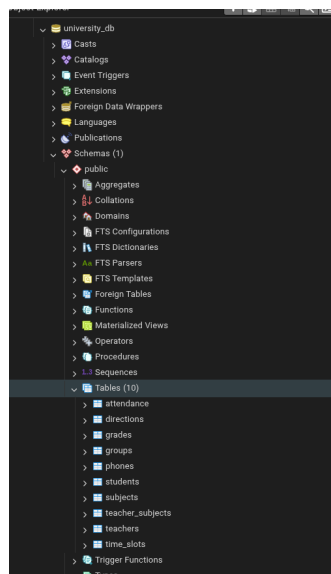


Рисунок 2 — pgAdmin: подключение к university_db на VM11

```

db1@db-server-11:~$ # список таблиц
PGPASSWORD=1 psql -h 127.0.0.1 -U db1_user -d university_db -c "\dt"

# размер (сколько строк)
PGPASSWORD=1 psql -h 127.0.0.1 -U db1_user -d university_db -c "
SELECT 'students' AS t, count(*) FROM students
UNION ALL SELECT 'phones', count(*) FROM phones
UNION ALL SELECT 'groups', count(*) FROM groups
UNION ALL SELECT 'teachers', count(*) FROM teachers
UNION ALL SELECT 'subjects', count(*) FROM subjects
UNION ALL SELECT 'grades', count(*) FROM grades
UNION ALL SELECT 'attendance', count(*) FROM attendance
ORDER BY 1;"

# пример ФИО
PGPASSWORD=1 psql -h 127.0.0.1 -U db1_user -d university_db -c "
SELECT id, last_name, first_name, middle_name FROM students ORDER BY id LIMIT 10;"

List of relations
Schema | Name | Type | Owner
-----+-----+-----+-----
public | attendance | table | postgres
public | directions | table | postgres
public | grades | table | postgres
public | groups | table | postgres
public | phones | table | postgres
public | students | table | postgres
public | subjects | table | postgres
public | teacher_subjects | table | postgres
public | teachers | table | postgres
public | time_slots | table | postgres
(10 rows)

 t | count
-----+-----
 attendance | 485
  grades | 678
  groups | 15
  phones | 320
 students | 240
 subjects | 15
 teachers | 5
(7 rows)

 id | last_name | first_name | middle_name
-----+-----+-----+-----
 1 | Иванов | Иван | Иванович
 2 | Петров | Петр | Петрович
 3 | Сидоров | Сидор | Сидорович
 4 | Кузнецов | Алексей | Дмитриевич
 5 | Смирнов | Дмитрий | Александрович
 6 | Волков | Андрей | Сергеевич
 7 | Зайцев | Михаил | Олегович
 8 | Павлов | Николай | Викторович
 9 | Семенов | Евгений | Андреевич
10 | Голубев | Сергей | Павлович
(10 rows)

```

Рисунок 3 — терминал VM11: список таблиц / проверка схемы

```

# пример ФИО
PGPASSWORD=1 psql -h 127.0.0.1 -U db1_user -d university_db -c "
SELECT id, last_name, first_name, middle_name FROM students ORDER BY id LIMIT 10;"

List of relations
Schema | Name | Type | Owner
-----+-----+-----+-----
public | attendance | table | postgres
public | directions | table | postgres
public | grades | table | postgres
public | groups | table | postgres
public | phones | table | postgres
public | students | table | postgres
public | subjects | table | postgres
public | teacher_subjects | table | postgres
public | teachers | table | postgres
public | time_slots | table | postgres
(10 rows)

 t | count
-----+-----
 attendance | 485
  grades | 678
  groups | 15
  phones | 320
 students | 240
 subjects | 15
 teachers | 5
(7 rows)

 id | last_name | first_name | middle_name
-----+-----+-----+-----
 1 | Иванов | Иван | Иванович
 2 | Петров | Петр | Петрович
 3 | Сидоров | Сидор | Сидорович
 4 | Кузнецов | Алексей | Дмитриевич
 5 | Смирнов | Дмитрий | Александрович
 6 | Волков | Андрей | Сергеевич
 7 | Зайцев | Михаил | Олегович
 8 | Павлов | Николай | Викторович
 9 | Семенов | Евгений | Андреевич
10 | Голубев | Сергей | Павлович
(10 rows)
db1@db-server-11:~$

```

Рисунок 4 — терминал VM11: выборка студентов с ФИО

```

serafim@pop-os:~$ # список ВМ
virsh --connect qemu:///system list --all

# снимок VM11
virsh --connect qemu:///system snapshot-create-as ubuntu24.04-11 \
before-lab4 \
"полная university_db после лабы 1" \
--atomic

# проверка
virsh --connect qemu:///system snapshot-list ubuntu24.04-11
Id Name State
-----
1 ubuntu24.04-2 running
2 ubuntu24.04 paused
4 ubuntu24.04-11 running

Domain snapshot before-lab4 created
Name Creation Time State
-----
before-lab4 2026-09-17 20:27:24 +0300 running

serafim@pop-os:~$

```

Рисунок 5 — скрин содержимого БД на VM11

```
serafim@pop-os:~$ virsh --connect qemu:///system snapshot-list ubuntu24.04-11
Name          Creation Time          State
-----
before-lab4    2026-09-17 20:27:24 +0300  running
```

Рисунок 6 — дополнительная проверка состояния VM11

3. Часть 1. Снимок и streaming-зеркало VM11 → VM2

Идея части 1: зафиксировать «эталон» полной БД снимком libvirt, затем получить на второй ВМ физическую копию PostgreSQL, которая непрерывно догоняет primary по WAL (Write-Ahead Log). На standby можно выполнять SELECT (hot_standby), но нельзя писать — это и есть «зеркало» из ТЗ.

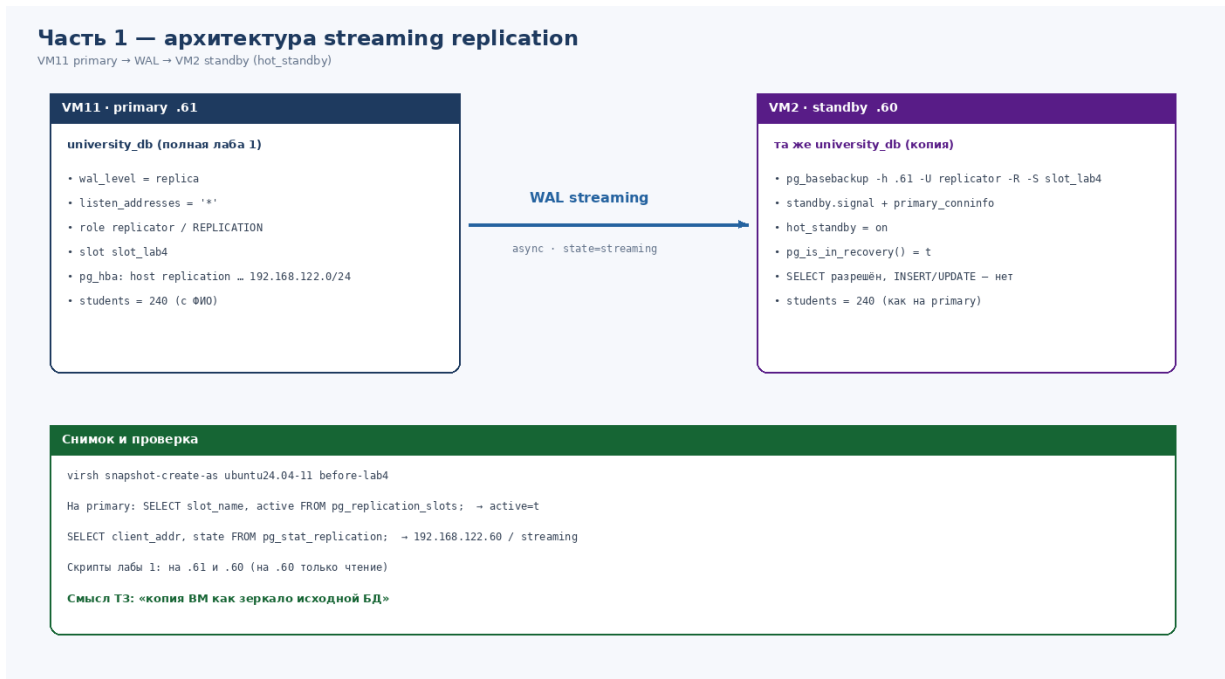


Рисунок U1 — UML/архитектура части 1: primary → WAL streaming → standby

3.1. Снимок before-lab4 (на хосте Pop!_OS)

Снимок делается не внутри SSH-сессии гостя, а у гипервизора: virsh / Virtual Machine Manager. Имя домена — ubuntu24.04-11.

```
virsh --connect qemu:///system snapshot-create-as ubuntu24.04-11 \
before-lab4 \
"полная university_db после лабы 1" \
--atomic
```

```
virsh --connect qemu:///system snapshot-list ubuntu24.04-11
```

Пояснение. snapshot-create-as сохраняет состояние диска ВМ. Имя before-lab4 нужно для части 2: откатом вернёмся к полной БД до разреза. --atomic делает снимок атомарно. Проверка snapshot-list должна показывать before-lab4.

```
serafim@pop-os:~$ virsh --connect qemu:///system snapshot-list ubuntu24.04-11
Name                               Creation Time             State
-----
before-lab4  2026-09-17 20:27:24 +0300  running
```

Рисунок 7 — virsh snapshot-list: before-lab4

3.2. Шаг А — настройка primary на VM11

Вход: ssh db1@192.168.122.61, пароль 1, hostname = db-server-11.

A1. Конфиг репликации

```
sudo tee /etc/postgresql/16/main/conf.d/99-replication.conf >/dev/null <<'EOF'
listen_addresses = '*'
wal_level = replica
max_wal_senders = 10
max_replication_slots = 10
hot_standby = on
wal_keep_size = 256MB
EOF
```

```
sudo cat /etc/postgresql/16/main/conf.d/99-replication.conf
```

Пояснение параметров. `listen_addresses='*'` — PostgreSQL слушает не только `localhost`, иначе VM2 по сети не подключится. `wal_level=replica` — в WAL достаточно информации для физической реплики. `max_wal_senders / max_replication_slots` — лимиты отправителей WAL и слотов. `hot_standby=on` — на standby разрешены читающие запросы. `wal_keep_size` — запас WAL на диске primary, пока standby догоняет.

A2. Доступ из сети BM (pg_hba.conf)

```
sudo bash -c 'grep -q "lab4 replication" /etc/postgresql/16/main/pg_hba.conf || cat >> /etc/postgresql/16/main/pg_hba.conf <<EOF
```

```
# lab4 replication / app
host replication replicator 192.168.122.0/24 scram-sha-256
host all all 192.168.122.0/24 scram-sha-256
EOF'
```

Пояснение. `pg_hba.conf` решает, кому можно подключаться. Строка `replication` + пользователь `replicator` разрешает именно поток репликации/basebackup из подсети libvirt `192.168.122.0/24`. Вторая строка — обычные клиентские подключения (удобно для проверок с хоста).

```
db1@db-server-11:~$ echo 1 | sudo -S bash -c 'grep -q "lab4 replication" /etc/postgresql/16/main/pg_hba.conf || cat >> /etc/postgresql/16/main/pg_hba.conf <<EOF
# lab4 replication / app
host replication replicator 192.168.122.0/24 scram-sha-256
host all all 192.168.122.0/24 scram-sha-256
EOF'
db1@db-server-11:~$ echo 1 | sudo -S -u postgres psql <<'SQL'
DO $$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'replicator') THEN
    CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD '1';
  ELSE
    ALTER ROLE replicator WITH REPLICATION LOGIN PASSWORD '1';
  END IF;
END IF;
END$$;

SELECT pg_create_physical_replication_slot('slot_lab4')
WHERE NOT EXISTS (
  SELECT 1 FROM pg_replication_slots WHERE slot_name = 'slot_lab4'
);
SQL
DO
  pg_create_physical_replication_slot
  (slot_lab4,)
(1 row)
```

Рисунок 8 — настройка `pg_hba` и создание роли/слота (терминал VM11)

A3. Роль `replicator` и слот `slot_lab4`

```
sudo -u postgres psql <<'SQL'
DO $$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'replicator') THEN
    CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD '1';
  ELSE
    ALTER ROLE replicator WITH REPLICATION LOGIN PASSWORD '1';
  END IF;
END IF;
END$$;

SELECT pg_create_physical_replication_slot('slot_lab4')
WHERE NOT EXISTS (
  SELECT 1 FROM pg_replication_slots WHERE slot_name = 'slot_lab4'
);
SQL
```

Пояснение. `replicator` — отдельная учётка с правом `REPLICATION` (не для меню лабы 1). Физический слот `slot_lab4` «закрепляет» нужный WAL за зеркалом: primary не удалит сегменты, пока standby их не подтвердил. Это важнее простого `wal_keep_size` при паузах standby.

```
db1@db-server-11:~$ sudo cat /etc/postgresql/16/main/conf.d/99-replication.conf
listen_addresses = '*'
wal_level = replica
max_wal_senders = 10
max_replication_slots = 10
hot_standby = on
wal_keep_size = 256MB
db1@db-server-11:~$ echo 1 | sudo -S bash -c 'grep -q "lab4 replication" /etc/postgresql/16/main/pg_hba.conf || cat >> /etc/postgresql/16/main/pg_hba.conf <<EOF
# lab4 replication / app
host replication replicator 192.168.122.0/24 scram-sha-256
host all all 192.168.122.0/24 scram-sha-256
EOF'
```

Рисунок 9 — результат создания `slot_lab4`

A4. Перезапуск и проверка primary

```
sudo systemctl restart postgresql
sleep 2
sudo -u postgres psql -c "SHOW wal_level;"
sudo -u postgres psql -c "SELECT slot_name, active FROM pg_replication_slots;"
```

Ожидание. wal_level = replica. Слот slot_lab4 есть; active пока может быть f — зеркало ещё не подключено.

```
db1@db-server-11:~$ echo 1 | sudo -S -u postgres psql <<'SQL'
DO $$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'replicator') THEN
    CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD '1';
  ELSE
    ALTER ROLE replicator WITH REPLICATION LOGIN PASSWORD '1';
  END IF;
END$$;

SELECT pg_create_physical_replication_slot('slot_lab4')
WHERE NOT EXISTS (
  SELECT 1 FROM pg_replication_slots WHERE slot_name = 'slot_lab4'
);
SQL
DO
  pg_create_physical_replication_slot
-----
(slot_lab4,)
(1 row)

db1@db-server-11:~$ echo 1 | sudo -S systemctl restart postgresql
sleep 2
echo 1 | sudo -S -u postgres psql -c "SHOW wal_level;"
echo 1 | sudo -S -u postgres psql -c "SELECT slot_name, active FROM pg_replication_slots;"
wal_level
-----
replica
(1 row)

slot_name | active
-----+-----
slot_lab4 | f
(1 row)
```

Рисунок 10 — после restart: wal_level=replica, slot_lab4 (active=f до basebackup)

3.3. Шаг В — зеркало на VM2

Вход: ssh db1@192.168.122.60, hostname = db-server-2.

B1-B2. Остановка Postgres и полная очистка PGDATA

```
sudo systemctl stop postgresql
```

```
# недостаточно: rm .../* не удаляет скрытые файлы
sudo ls -la /var/lib/postgresql/16/main
sudo find /var/lib/postgresql/16/main -mindepth 1 -delete
sudo ls -la /var/lib/postgresql/16/main
```

Пояснение. Пока сервис запущен, чистить data directory нельзя. Ошибка «directory exists but is not empty» при basebackup как раз из-за остатков (в т.ч. скрытых). find -mindepth 1 -delete очищает каталог полностью.

```
db1@db-server-11:~$ echo 1 | sudo -S -u postgres psql <<'SQL'
DO $$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'replicator') THEN
    CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD '1';
  ELSE
    ALTER ROLE replicator WITH REPLICATION LOGIN PASSWORD '1';
  END IF;
END$$;

SELECT pg_create_physical_replication_slot('slot_lab4')
WHERE NOT EXISTS (
  SELECT 1 FROM pg_replication_slots WHERE slot_name = 'slot_lab4'
);
SQL
DO
  pg_create_physical_replication_slot
-----
(slot_lab4,)
(1 row)

db1@db-server-11:~$ echo 1 | sudo -S systemctl restart postgresql
sleep 2
echo 1 | sudo -S -u postgres psql -c "SHOW wal_level;"
echo 1 | sudo -S -u postgres psql -c "SELECT slot_name, active FROM pg_replication_slots;"
wal_level
-----
replica
(1 row)

slot_name | active
-----+-----
slot_lab4 | f
(1 row)
```

Рисунок 11 — очистка /var/lib/postgresql/16/main на VM2

B3. pg_basebackup с VM11

```

sudo -u postgres PGPASSWORD=1 pg_basebackup \
-h 192.168.122.61 \
-p 5432 \
-U replicator \
-D /var/lib/postgresql/16/main \
-Fp -Xs -P -R \
-S slot_lab4

```

Разбор флагов. -h 192.168.122.61 — источник (VM11). -U replicator — роль репликации. -D .../main — куда класть файлы кластера. -Fp — plain format (обычный каталог данных). -Xs — стримить WAL во время копирования. -P — прогресс. -R — сразу создать standby.signal и primary_conninfo (режим standby). -S slot_lab4 — использовать созданный слот. По окончании видно 100% tablespace.

```

db1@db-server-2:~$ sudo -u postgres PGPASSWORD=1 pg_basebackup \
-h 192.168.122.61 \
-p 5432 \
-U replicator \
-D /var/lib/postgresql/16/main \
-Fp -Xs -P -R \
-S slot_lab4
31612/31612 kB (100%), 1/1 tablespace
db1@db-server-2:~$

```

Рисунок 12 — успешный pg_basebackup (100%)

В4-В5. Права, hot_standby, старт, проверка

```

sudo chown -R postgres:postgres /var/lib/postgresql/16/main
sudo chmod 700 /var/lib/postgresql/16/main

```

```

sudo tee /etc/postgresql/16/main/conf.d/99-standby.conf >/dev/null <<'EOF'
hot_standby = on
EOF

```

```

sudo systemctl start postgresql
sleep 2
sudo -u postgres psql -c "SELECT pg_is_in_recovery();"
sudo -u postgres psql -d university_db -c "SELECT count(*) FROM students;"

```

Ожидание. pg_is_in_recovery() = t (это standby). count(students) = 240 — как на primary. chmod 700 обязателен: PostgreSQL не стартует с «открытым» PGDATA.

```

db1@db-server-2:~$ sudo chown -R postgres:postgres /var/lib/postgresql/16/main
db1@db-server-2:~$ sudo chmod 700 /var/lib/postgresql/16/main
db1@db-server-2:~$ sudo systemctl start postgresql
db1@db-server-2:~$ sleep 2
db1@db-server-2:~$ sudo -u postgres psql -c "SELECT pg_is_in_recovery();"
pg_is_in_recovery
-----
t
(1 row)

db1@db-server-2:~$ sudo -u postgres psql -d university_db -c "SELECT count(*) FROM students;"
count
-----
240
(1 row)

```

Рисунок 13 — VM2: recovery=t, students=240

3.4. Шаг С — проверка streaming на primary

```

# снова на VM11
ssh db1@192.168.122.61

```

```

sudo -u postgres psql -c "SELECT slot_name, active FROM pg_replication_slots;"
sudo -u postgres psql -c "SELECT client_addr, state, sync_state FROM pg_stat_replication;"
sudo -u postgres psql -d university_db -c "SELECT count(*) FROM students;"

```

Смысл проверки. active=t значит слот занят живым standby. pg_stat_replication показывает клиента .60, state=streaming, sync_state=async (для учебной лабы async достаточно). COUNT на primary тоже

240. Primary непрерывно шлёт WAL на зеркало.

```
db1@db-server-11:~$ sudo -u postgres psql -c "SELECT slot_name, active FROM pg_replication_slots;"
sudo -u postgres psql -c "SELECT client_addr, state, sync_state FROM pg_stat_replication;"
sudo -u postgres psql -d university_db -c "SELECT count(*) FROM students;"
[sudo] password for db1:
 slot_name | active
-----+-----
 slot_lab4 | t
(1 row)

 client_addr | state | sync_state
-----+-----+-----
 192.168.122.60 | streaming | async
(1 row)

 count
-----
    240
(1 row)
```

Рисунок 14 — VM11: slot active, streaming → .60, students=240

BM	IP	После части 2
VM11	.61	Университет: группы, предметы, оценки, посещаемость... без ФИО/ телефонов (или ФИО обнулены)
VM2	.60	Только личное: студенты (ФИО, адрес...) + телефоны

Рисунок 15 — дополнительный фрагмент проверки части 1

4. Часть 2. Разрез данных и distributed-клиент

После части 1 обе VM содержат одну и ту же полную БД (primary + standby). В части 2 данные специально разделяются: университетские сущности остаются на VM11 (без ФИО/телефонов), личный контур — только на VM2. Клиент на Python открывает два соединения и склеивает ответ по общему ключу students.id.

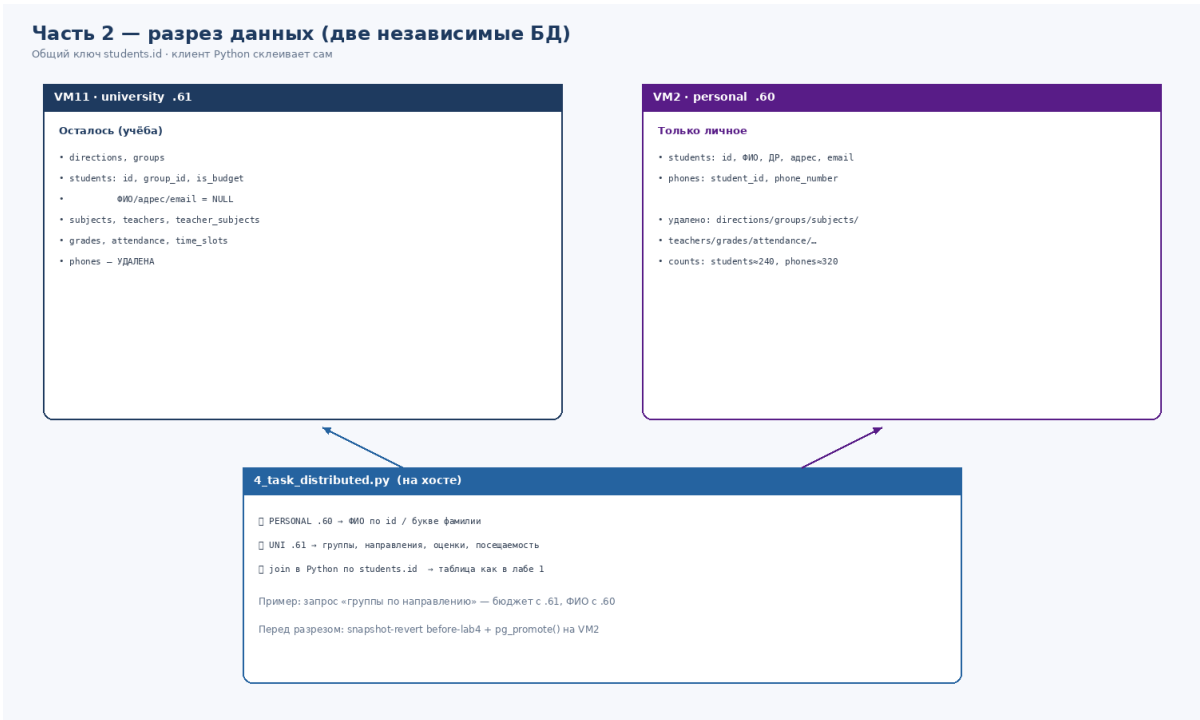


Рисунок U2 — UML/архитектура части 2: разрез university / personal

Чем отличается от части 1		
	Часть 1	Часть 2
Данные	Одинаковые на обеих	Разные: учёба / личное
Связь VM	Streaming replication	Два независимых сервера
Клиент	Один хост (как лаба 1)	Два PGHOST / два соединения

Рисунок 16 — чем часть 2 отличается от части 1

4.1. Откат VM11 к снимку before-lab4

По ТЗ нужно вернуться к исходному состоянию полной БД. Скрины части 1 (streaming) сохранены заранее — откат «ломает» текущее зеркало.

```
# на хосте pop-os
virsh --connect qemu:///system snapshot-list ubuntu24.04-11

virsh --connect qemu:///system snapshot-revert ubuntu24.04-11 before-lab4
# Domain snapshot before-lab4 reverted

virsh --connect qemu:///system start ubuntu24.04-11
sleep 15
ssh db1@192.168.122.61

hostname
sudo -u postgres psql -c "SELECT pg_is_in_recovery();"
sudo -u postgres psql -d university_db -c "
SELECT count(*) FROM students;
SELECT count(*) FILTER (WHERE last_name IS NOT NULL) AS with_fio FROM students;"
```

Пояснение. snapshot-revert возвращает диск VM к моменту снимка: снова полная university_db, без текущей конфигурации репликации «поверх» снимка. Ожидание: recovery=f, 240 студентов, with_fio=240. VM2 отдельно не откатывали (снимка могло не быть): у неё остаётся полная копия данных, но связь streaming после отката primary уже неактуальна.

```

Domain snapshot before-lab4 reverted

serafim@pop-os:~$ virsh --connect qemu:///system start ubuntu24.04-11
sleep 15
ssh db1@192.168.122.61
error: Domain is already active

db1@192.168.122.61's password:
Welcome to Ubuntu 24.04.5 LTS (GNU/Linux 6.8.0-139-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Thu Sep 17 04:41:07 PM UTC 2026

System load:  1.44           Processes:           201
Usage of /:   56.3% of 24.44GB Users logged in:       0
Memory usage: 67%          IPv4 address for enp1s0: 192.168.122.61
Swap usage:   12%

 * Canonical Workshop gives developers fast, composable, reproducible, and
   secure developer environments that are perfect for agentic workflows.

   https://ubuntu.com/workshop

Expanded Security Maintenance for Applications is not enabled.

1 update can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check
your Internet connection or proxy settings

Last login: Thu Sep 17 16:41:08 2026 from 192.168.122.1
db1@db-server-11:~$

```

Рисунок 17 — откат / состояние после revert (скрин из журнала выполнения)

```

db1@db-server-11:~$ hostname
sudo -u postgres psql -c "SELECT pg_is_in_recovery();"
sudo -u postgres psql -d university_db -c "SELECT count(*) FROM students; S
ELECT count(*) FILTER (WHERE last_name IS NOT NULL) AS with_fio FROM studen
ts;"
db-server-11
[sudo] password for db1:
pg_is_in_recovery
-----
f
(1 row)

count
-----
240
(1 row)

with_fio
-----
240
(1 row)
db1@db-server-11:~$

```

Рисунок 18 — проверка VM11 после отката

4.2. Promote VM2 — сделать самостоятельной БД

На VM2 всё ещё лежат файлы standby. Primary уже «другой» после отката. Нужно превратить бывшее зеркало в обычный writable primary (полная копия данных), чтобы потом безопасно вырезать университетские таблицы.

```
ssh db1@192.168.122.60
```

```

sudo -u postgres psql -c "SELECT pg_promote();"
# при необходимости:
sudo rm -f /var/lib/postgresql/16/main/standby.signal
sudo systemctl restart postgresql
sleep 3

```

```

sudo -u postgres psql -c "SELECT pg_is_in_recovery();"
sudo -u postgres psql -d university_db -c "
SELECT count(*) AS students,
       count(*) FILTER (WHERE last_name IS NOT NULL) AS with_fio
FROM students;"

```

Пояснение. `pg_promote()` выводит инстанс из recovery. После этого `recovery=f`, данные полные (240/240). Обе ВМ — независимые primary с одинаковым содержимым; дальше режим каждую

```

Last login: Thu Sep 17 17:59:21 2026 from 192.168.122.1
db1@db-server-2:~$ sudo -u postgres psql -c "SELECT pg_promote();"
[sudo] password for db1:
pg_promote
-----
t
(1 row)

db1@db-server-2:~$ sudo -u postgres psql -c "SELECT pg_is_in_recovery();"
sudo -u postgres psql -d university_db -c "
SELECT count(*) AS students,
       count(*) FILTER (WHERE last_name IS NOT NULL) AS with_fio
FROM students;"
pg_is_in_recovery
-----
f
(1 row)

students | with_fio
-----+-----
240      |      240
(1 row)

```

Рисунок 19 — promote VM2: recovery=f, 240 с ФИО

4.3. Разрез на VM2 — только личные данные (полный SQL)

Идея: сохранить ФИО/адрес/email и телефоны во временные таблицы, удалить всё университетское, затем заново создать только students и phones и вернуть данные.

```

sudo -u postgres psql -d university_db <<'SQL'
BEGIN;

-- 1) сохранить личное во временные таблицы
DROP TABLE IF EXISTS _personal_students CASCADE;
CREATE TABLE _personal_students AS
SELECT id, last_name, first_name, middle_name, birth_date,
       city, street, house_number, email
FROM students;

DROP TABLE IF EXISTS _personal_phones CASCADE;
CREATE TABLE _personal_phones AS
SELECT id, student_id, phone_number FROM phones;

-- 2) удалить университетское (+ старые students/phones)
DROP TABLE IF EXISTS attendance CASCADE;
DROP TABLE IF EXISTS grades CASCADE;
DROP TABLE IF EXISTS teacher_subjects CASCADE;
DROP TABLE IF EXISTS subjects CASCADE;
DROP TABLE IF EXISTS teachers CASCADE;
DROP TABLE IF EXISTS time_slots CASCADE;
DROP TABLE IF EXISTS phones CASCADE;
DROP TABLE IF EXISTS students CASCADE;
DROP TABLE IF EXISTS groups CASCADE;
DROP TABLE IF EXISTS directions CASCADE;

-- 3) заново создать только личные таблицы
CREATE TABLE students (
  id INTEGER PRIMARY KEY,
  last_name VARCHAR(100) NOT NULL,
  first_name VARCHAR(100) NOT NULL,
  middle_name VARCHAR(100),
  birth_date DATE NOT NULL,
  city VARCHAR(100) NOT NULL,
  street VARCHAR(100) NOT NULL,
  house_number VARCHAR(20) NOT NULL,
  email VARCHAR(100) UNIQUE
);
INSERT INTO students SELECT * FROM _personal_students;

CREATE TABLE phones (
  id INTEGER PRIMARY KEY,
  student_id INTEGER NOT NULL REFERENCES students(id) ON DELETE CASCADE,
  phone_number VARCHAR(20) NOT NULL
);
INSERT INTO phones SELECT * FROM _personal_phones;

DROP TABLE _personal_students;
DROP TABLE _personal_phones;

GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO db1_user;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO db1_user;

```

```
COMMIT;
```

```
\dt
SELECT 'students' AS t, count(*) FROM students
UNION ALL SELECT 'phones', count(*) FROM phones;
SQL
```

Пояснение. Временные таблицы `_personal_*` — страховка: сначала копируем личное, потом DROP всего остального. Новые `students/phones` содержат только персональный контур. Ожидание: в `\dt` две таблицы; counts ≈ 240 и 320 . `id` студентов не менялись — это общий ключ с `VM11`.

```
CREATE TABLE phones (
SQL> ALL SELECT 'phones', count(*) FROM phones; public TO db1_user
BEGIN
NOTICE: table "_personal_students" does not exist, skipping
DROP TABLE
SELECT 240
NOTICE: table "_personal_phones" does not exist, skipping
DROP TABLE
SELECT 320
DROP TABLE
DROP TABLE
DROP TABLE
DROP TABLE
DROP TABLE
DROP TABLE
DROP TABLE
DROP TABLE
DROP TABLE
DROP TABLE
CREATE TABLE
INSERT 0 240
CREATE TABLE
INSERT 0 320
DROP TABLE
DROP TABLE
GRANT
GRANT
COMMIT

      List of relations
Schema | Name   | Type  | Owner
-----+-----+-----+-----
public | phones | table | postgres
public | students | table | postgres
(2 rows)

 t | count
--+-----
students | 240
phones | 320
(2 rows)
```

Рисунок 20 — VM2 после разреза: только `students` и `phones` (240 / 320)

4.4. Разрез на VM11 — учёба без личного (полный SQL)

На `primary` оставляем направления/группы/оценки/посещаемость. Таблица `phones` удаляется (телефоны живут на `VM2`). Поля ФИО/адреса/email в `students` обнуляются, но строки и `id` сохраняются.

```
ssh db1@192.168.122.61
```

```
sudo -u postgres psql -d university_db <<'SQL'
BEGIN;
```

```
DROP TABLE IF EXISTS phones CASCADE;
```

```
ALTER TABLE students
  ALTER COLUMN last_name DROP NOT NULL,
  ALTER COLUMN first_name DROP NOT NULL,
  ALTER COLUMN birth_date DROP NOT NULL,
  ALTER COLUMN city DROP NOT NULL,
  ALTER COLUMN street DROP NOT NULL,
  ALTER COLUMN house_number DROP NOT NULL;
```

```
UPDATE students SET
  last_name = NULL,
  first_name = NULL,
  middle_name = NULL,
  birth_date = NULL,
  city = NULL,
  street = NULL,
  house_number = NULL,
  email = NULL;
```

```
COMMIT;
```

```
\dt
SELECT count(*) AS students,
       count(*) FILTER (WHERE last_name IS NULL) AS anon
FROM students;
SELECT count(*) FROM grades;
SELECT count(*) FROM groups;
SQL
```

Пояснение. ALTER ... DROP NOT NULL нужен, иначе UPDATE в NULL упадёт на ограничениях.
Ожидание: phones нет; students=240 и все anon; grades≈678, groups=15. Так выполняется пункт Т3 «на первой ВМ — без личных данных / личное на второй».

```

SELECT count(*) FROM students;
count(*) FILTER (WHERE last_name IS NULL) AS anon
FROM students;
SELECT count(*) FROM grades;
SELECT count(*) FROM groups;
SQL
[sudo] password for db1:
BEGIN
DROP TABLE
ALTER TABLE
UPDATE 240
COMMIT

```

Schema	Name	Type	Owner
public	attendance	table	postgres
public	directions	table	postgres
public	grades	table	postgres
public	groups	table	postgres
public	students	table	postgres
public	subjects	table	postgres
public	teacher_subjects	table	postgres
public	teachers	table	postgres
public	time_slots	table	postgres

(9 rows)

```

students | anon
-----+-----
240 | 240
(1 row)

count
-----
678
(1 row)

count
-----
15
(1 row)

```

Рисунок 21 — VM11 после разреза: без phones, ФИО=NULL, учёба на месте

4.5. Клиент Python на две БД

Скрипт 4_task_distributed.py: UNI=192.168.122.61, PERSONAL=192.168.122.60. Для запроса «группы по направлению» группы/бюджет берутся с VM11, ФИО — с VM2, склейка в Python по id. В меню выводятся примеры направлений/букв.

```

cd "/home/serafim/Рабочий стол/РБД/Выполненные задания/К_сдаче_преподавателю/_агент_сборка/4_task"
source .venv/bin/activate
python 4_task_distributed.py
# Подключено: UNI= 192.168.122.61 PERSONAL= 192.168.122.60
# Выбор: 1 → направление (Enter или ввод строки) → таблица с ФИО

```

```

1. Списки групп по направлению
Доступные направления:
1. Информатика и вычислительная техника
2. Информационная безопасность
3. Прикладная математика и информатика
4. Программная инженерия
5. Системный анализ и управление

- Направление (скопируй строку целиком)
примеры: Информатика и вычислительная техника | Информационна
безопасность | Прикладная математика и информатика
[Enter = «Информатика и вычислительная техника»]
ввод: Информатика и вычислительная техника

```

Группа	ФИО	Тип
ИБТ-101	Богданов Роман Денисович	Бюджет
ИБТ-101	Виноградов Артём Игоревич	Бюджет
ИБТ-101	Волков Андрей Сергеевич	Внебюджет
ИБТ-101	Воробьев Максим Александрович	Внебюджет
ИБТ-101	Голубев Сергей Павлович	Бюджет
ИБТ-101	Зайцев Михаил Олегович	Бюджет
ИБТ-101	Иванов Иван Иванович	Бюджет
ИБТ-101	Кузнецов Алексей Дмитриевич	Бюджет
ИБТ-101	Михайлов Тимофей Андреевич	Бюджет
ИБТ-101	Павлов Николай Викторович	Бюджет
ИБТ-101	Петров Петр Петрович	Бюджет

Рисунок 22 — меню distributed-клиента (два хоста)

```
1. Списки групп по направлению
Доступные направления:
1. Информатика и вычислительная техника
2. Информационная безопасность
3. Прикладная математика и информатика
4. Программная инженерия
5. Системный анализ и управление

→ Направление (скопируй строку целиком)
примеры: Информатика и вычислительная техника | Информационна
я безопасность | Прикладная математика и информатика
[Enter = «Информатика и вычислительная техника»]
ввод: Информатика и вычислительная техника

+-----+
| Группа | ФИО | Тип |
+-----+
| ИВТ-101 | Богданов Роман Денисович | Бюджет |
+-----+
| ИВТ-101 | Виноградов Артём Игоревич | Бюджет |
+-----+
| ИВТ-101 | Волков Андрей Сергеевич | Внебюджет |
+-----+
| ИВТ-101 | Воробьев Максим Александрович | Внебюджет |
+-----+
| ИВТ-101 | Голубев Сергей Павлович | Бюджет |
+-----+
| ИВТ-101 | Зайцев Михаил Олегович | Бюджет |
+-----+
| ИВТ-101 | Иванов Иван Иванович | Бюджет |
+-----+
| ИВТ-101 | Кузнецов Алексей Дмитриевич | Бюджет |
+-----+
| ИВТ-101 | Михайлов Тимофей Андреевич | Бюджет |
+-----+
| ИВТ-101 | Павлов Николай Викторович | Бюджет |
+-----+
| ИВТ-101 | Петров Петр Петрович | Бюджет |
```

Рисунок 23 — результат: группы + ФИО (данные с двух ВМ)

5. Часть 3. postgres_fdw — «одна БД» для клиента

В части 2 склейку делал Python. В части 3 склейку делает сам PostgreSQL на VM11: через Foreign Data Wrapper видны таблицы VM2. Клиент подключается только к .61, а ФИО прозрачно читаются с .60.

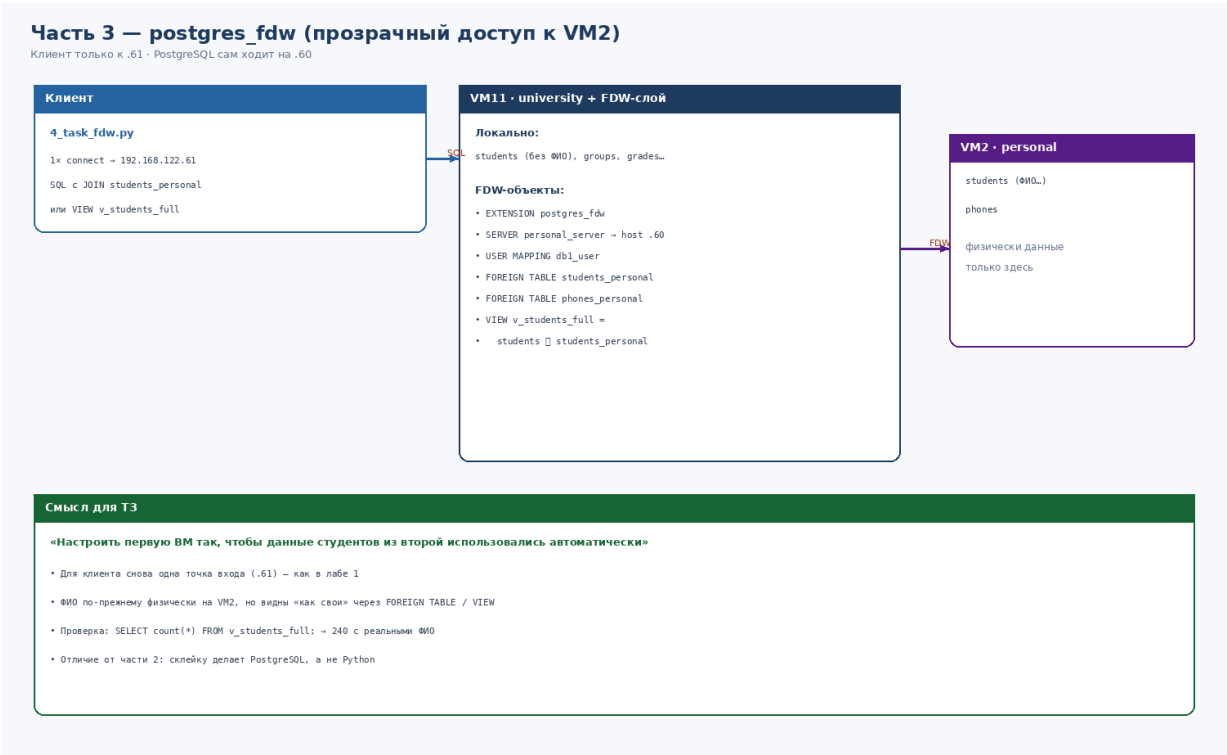


Рисунок U3 — UML/архитектура части 3: FDW и v_students_full

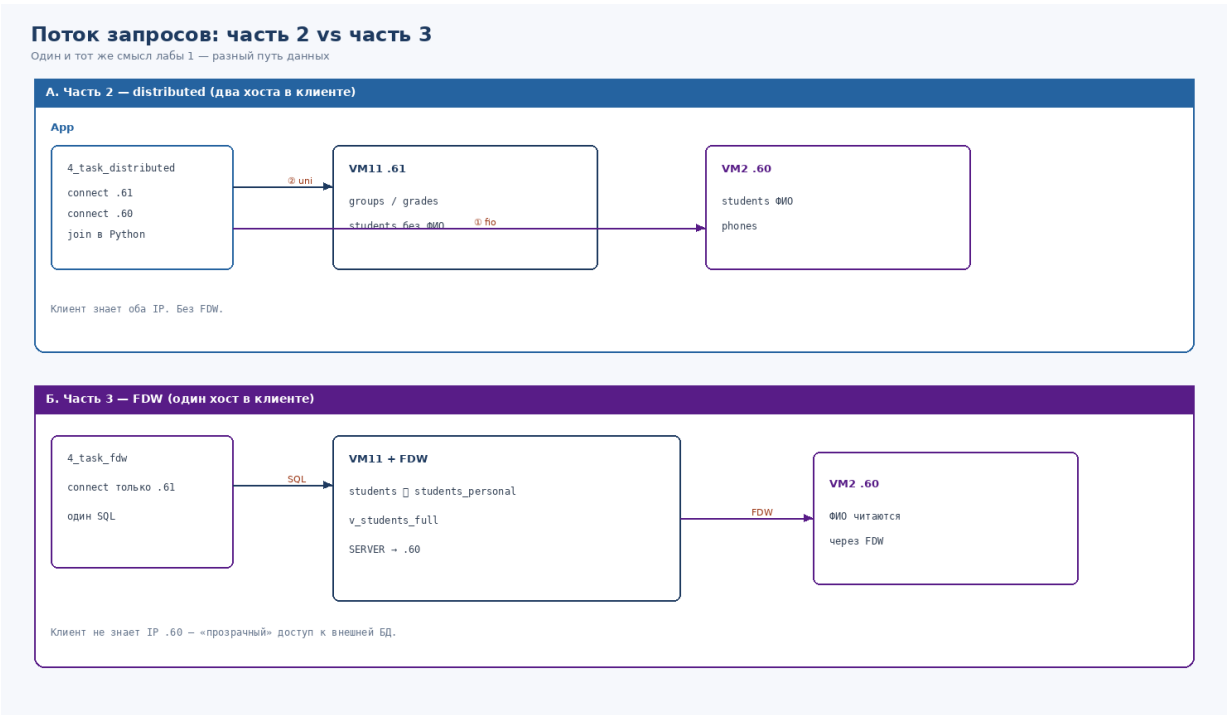


Рисунок U4 — поток запросов: часть 2 (два хоста) vs часть 3 (FDW)

Что получится

Объект на VM11	Смысл
<code>SERVER personal_server</code>	Ссылка на VM2 (.60)
<code>students_personal</code>	Внешняя таблица = <code>students</code> на VM2
<code>phones_personal</code>	Внешняя таблица = <code>phones</code> на VM2
<code>v_students_full</code>	View: локальный студент + ФИО с VM2

Рисунок 24 — какие объекты появятся на VM11

	Часть 2	Часть 3
Кто склеивает	Python	PostgreSQL (FDW + view)
Куда подключается клиент	два хоста	в основном один (.61)
Где лежат ФИО физически	только VM2	всё ещё только VM2, но видны «как свои» на VM11

Рисунок 25 — отличие части 3 от части 2

5.1. Полная настройка FDW на VM11

Ниже — полный блок, фактически выполненный на db-server-11: расширение, сервер, user mapping, внешние таблицы, представление, права.

```
ssh db1@192.168.122.61

sudo -u postgres psql -d university_db <<'SQL'
CREATE EXTENSION IF NOT EXISTS postgres_fdw;

DROP USER MAPPING IF EXISTS FOR db1_user SERVER personal_server;
DROP USER MAPPING IF EXISTS FOR postgres SERVER personal_server;
DROP SERVER IF EXISTS personal_server CASCADE;

CREATE SERVER personal_server
  FOREIGN DATA WRAPPER postgres_fdw
  OPTIONS (host '192.168.122.60', port '5432', dbname 'university_db');

CREATE USER MAPPING FOR db1_user
  SERVER personal_server
  OPTIONS (user 'db1_user', password '1');

CREATE USER MAPPING FOR postgres
  SERVER personal_server
  OPTIONS (user 'db1_user', password '1');

DROP FOREIGN TABLE IF EXISTS students_personal CASCADE;
DROP FOREIGN TABLE IF EXISTS phones_personal CASCADE;

CREATE FOREIGN TABLE students_personal (
  id INTEGER,
  last_name VARCHAR(100),
  first_name VARCHAR(100),
  middle_name VARCHAR(100),
  birth_date DATE,
  city VARCHAR(100),
  street VARCHAR(100),
  house_number VARCHAR(20),
  email VARCHAR(100)
) SERVER personal_server
  OPTIONS (schema_name 'public', table_name 'students');

CREATE FOREIGN TABLE phones_personal (
  id INTEGER,
  student_id INTEGER,
  phone_number VARCHAR(20)
) SERVER personal_server
  OPTIONS (schema_name 'public', table_name 'phones');

CREATE OR REPLACE VIEW v_students_full AS
SELECT
  s.id,
  sp.last_name,
  sp.first_name,
  sp.middle_name,
  sp.birth_date,
  sp.city,
  sp.street,
  sp.house_number,
  sp.email,
```

```

s.group_id,
s.is_budget
FROM students s
JOIN students_personal sp ON sp.id = s.id;

GRANT USAGE ON FOREIGN SERVER personal_server TO db1_user;
GRANT SELECT ON students_personal TO db1_user;
GRANT SELECT ON phones_personal TO db1_user;
GRANT SELECT ON v_students_full TO db1_user;
SQL

```

Пояснение объектов. CREATE EXTENSION postgres_fdw — модуль доступа к другой PostgreSQL. SERVER personal_server — «куда звонить» (.60, university_db). USER MAPPING — под каким логином/паролем ходить на VM2. FOREIGN TABLE students_personal / phones_personal — локальные «окна» на удалённые таблицы. VIEW v_students_full — удобная склейка локального students (group_id, is_budget) с ФИО с VM2 по id. Для приложения это выглядит как обычный SELECT.

```

db1@db-server-11:~$ sudo -u postgres psql -d university_db <<'SQL'
CREATE EXTENSION IF NOT EXISTS postgres_fdw;

DROP USER MAPPING IF EXISTS FOR db1_user SERVER personal_server;
DROP USER MAPPING IF EXISTS FOR postgres SERVER personal_server;
DROP SERVER IF EXISTS personal_server CASCADE;

-- «МОСТ» к VM2
CREATE SERVER personal_server
  FOREIGN DATA WRAPPER postgres_fdw
  OPTIONS (host '192.168.122.60', port '5432', dbname 'university_db');

-- под каким логином ходим на VM2
CREATE USER MAPPING FOR db1_user
  SERVER personal_server
  OPTIONS (user 'db1_user', password '1');

CREATE USER MAPPING FOR postgres
  SERVER personal_server
  OPTIONS (user 'db1_user', password '1');
SQL
CREATE EXTENSION
NOTICE: server "personal_server" does not exist, skipping
DROP USER MAPPING
NOTICE: server "personal_server" does not exist, skipping
DROP USER MAPPING
NOTICE: server "personal_server" does not exist, skipping
DROP SERVER
CREATE SERVER
CREATE USER MAPPING
CREATE USER MAPPING

```

Рисунок 26 — создание FDW-объектов (фрагмент терминала)

```

middle_name VARCHAR(100),
birth_date DATE,
city VARCHAR(100),
street VARCHAR(100),
house_number VARCHAR(20),
email VARCHAR(100)
) SERVER personal_server
  OPTIONS (schema_name 'public', table_name 'students');

CREATE FOREIGN TABLE phones_personal (
  id INTEGER,
  student_id INTEGER,
  phone_number VARCHAR(20)
) SERVER personal_server
  OPTIONS (schema_name 'public', table_name 'phones');

CREATE OR REPLACE VIEW v_students_full AS
SELECT
  s.id,
  sp.last_name,
  sp.first_name,
  sp.middle_name,
  sp.birth_date,
  sp.city,
  sp.street,
  sp.house_number,
  sp.email,
  s.group_id,
  s.is_budget
FROM students s
JOIN students_personal sp ON sp.id = s.id;

GRANT USAGE ON FOREIGN SERVER personal_server TO db1_user;
SQL> SELECT ON v_students_full TO db1_user;
NOTICE: foreign table "students_personal" does not exist, skipping
DROP FOREIGN TABLE
NOTICE: foreign table "phones_personal" does not exist, skipping
DROP FOREIGN TABLE
CREATE FOREIGN TABLE
CREATE FOREIGN TABLE
CREATE VIEW
GRANT
GRANT
GRANT
GRANT

```

Рисунок 27 — CREATE FOREIGN TABLE / VIEW / GRANT

5.2. Проверка, что FDW читает VM2

```

sudo -u postgres psql -d university_db -c "

```

```
SELECT count(*) FROM students_personal;
SELECT count(*) FROM v_students_full;
SELECT id, last_name, first_name FROM v_students_full ORDER BY id LIMIT 5;
"
```

Ожидание. Оба COUNT = 240; в выборке реальные ФИО (Иванов, Петров...), хотя локальный students.ФИО = NULL. Значит строки действительно приходят с .60.

```
db1@db-server-11:~$ sudo -u postgres psql -d university_db -c "
SELECT count(*) FROM students_personal;
SELECT count(*) FROM v_students_full;
SELECT id, last_name, first_name FROM v_students_full ORDER BY id L
IMIT 5;
"
count
-----
240
(1 row)

count
-----
240
(1 row)

 id | last_name | first_name
-----+-----
  1 | Иванов    | Иван
  2 | Петров    | Петр
  3 | Сидоров   | Сидор
  4 | Кузнецов  | Алексей
  5 | Смирнов   | Дмитрий
(5 rows)
```

Рисунок 28 — v_students_full: 240 строк и ФИО через FDW

5.3. Клиент с одним подключением

```
cd "/home/serafim/Рабочий стол/РБД/Выполненные задания/К_сдаче_преподавателю/_агент_сборка/4_task"
source .venv/bin/activate
python 4_task_fdw.py
# FDW-клиент → 192.168.122.61 (личные данные тянутся с .60 через FDW)
# запросы 1 / 2 / 14 используют JOIN students_personal или эквивалент
```

Пояснение. В отличие от части 2, здесь один psycopg2.connect на .61. PostgreSQL сам ходит на VM2 по FDW. Для защиты это демонстрация пункта ТЗ «данные студентов из второй VM автоматически используются в первой».

```
sk/4_task_distributed.py", line 245, in main
    choice = input("Выбор: ").strip()
KeyboardInterrupt

(.venv) serafim@pop-os:~/Рабочий стол/РБД/Выполненные задания/4
_task$ cd "/home/serafim/Рабочий стол/РБД/Выполненные задания/4
_task"
source .venv/bin/activate
python 4_task_fdw.py
✓ FDW-клиент → 192.168.122.61 (личные данные тянутся с .60 че
рез FDW)

=====
ЗАДАНИЕ 4 · FDW-КЛИЕНТ (одно подключение)
[1] Группы по направлению (FDW)
[2] Фамилия на букву (FDW)
[4] Возраст (FDW)
[6] Кол-во по направлениям
[12] Средняя оценка
[14] Отличники (FDW ФИО)
[0] Выход
примеры выбора: 1 | 2 | 14 | 0
Выбор: 
```

Рисунок 29 — меню FDW-клиента (одно подключение)

```
(.venv) serafim@pop-os: ~/Рабочий стол/РБД/Выполненные задания/4
_task$ cd ~/home/serafim/Рабочий стол/РБД/Выполненные задания/4_task
source .venv/bin/activate
python 4_task_fdw.py
✓ FDW-клиент -- 192.168.122.61 (личные данные тянутся с .60 через FDW)

=====
ЗАДАНИЕ 4 - FDW-КЛИЕНТ (одно подключение)
[1] Группы по направлению (FDW)
[2] Фамилия на букву (FDW)
[4] Возраст (FDW)
[6] Кол-во по направлениям
[12] Средняя оценка
[14] Отличники (FDW ФИО)
[0] Выход
примеры выбора: 1 | 2 | 14 | 0
Выбор: 1

Группы по направлению (FDW)
Доступные направления:
1. Информатика и вычислительная техника
2. Информационная безопасность
3. Прикладная математика и информатика
4. Программная инженерия
5. Системный анализ и управление

-- Направление (скопируй строку целиком)
примеры: Информатика и вычислительная техника | Информационная безопасность | Прикладная математика и информатика
[Enter = «Информатика и вычислительная техника»]
ввод: Программная инженерия
+-----+
| Группа | ФИО | Тип |
+-----+
| ПИ-201 | Алексей Александр Петрович | Бюджет |
+-----+
| ПИ-201 | Белов Борис Иванович | Бюджет |
+-----+
| ПИ-201 | Васильев Виктор Сергеевич | Внебюджет |
+-----+
| ПИ-201 | Григорьев Глеб Андреевич | Бюджет |
+-----+
| ПИ-201 | Дмитриев Даниил Михайлович | Бюджет |
+-----+
```

Рисунок 30 — результат запроса через FDW (Программная инженерия, с ФИО)

Итог задания 4		
Часть	Суть	Статус
1	Снимок + зеркало VM11 → VM2	✓
2	Разрез + Python на две БД	✓
3	FDW на VM11 + клиент на .61	✓

Для сдачи сохрани скрины всех трёх частей. Если нужно — обновим PDF-отчёт под VM11

Рисунок 31 — итог трёх частей задания 4

6. Python-клиенты: устройство и запуск

Для проверки частей 2 и 3 написаны два учебных клиента на Python 3 (psycopg2 + tabulate). Они повторяют смысл запросов лабы 1, но учитывают распределённую схему. Код лежит в каталоге Выполненные задания/4_task/ и снабжён комментариями в самих файлах. Полные листинги обоих скриптов приведены в п. 6.5–6.6.

6.1. Общая подготовка окружения

```
cd "/home/serafim/Рабочий стол/РБД/Выполненные задания/К_сдаче_преподавателю/_агент_сборка/4_task"
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
# нужны: psycopg2-binary, tabulate
```

Пояснение. Виртуальное окружение изолирует зависимости. Оба скрипта рассчитаны на запуск с хоста Pop!_OS, пока ВМ доступны по 192.168.122.x. Учётка БД: db1_user / 1, база university_db.

6.2. 4_task_distributed.py — часть 2 (два подключения)

Скрипт открывает два соединения: UNI → VM11 (.61) и PERSONAL → VM2 (.60). Типичный запрос (№1 «группы по направлению») работает так: (1) с university читаются group_id / is_budget; (2) по списку id с personal читаются ФИО; (3) строки склеиваются в Python по students.id. Запросы 6 и 12 используют только university (ФИО не нужны). Запрос 2 наоборот сначала фильтрует фамилии на VM2, потом дотягивает группы с VM11.

```
source .venv/bin/activate
python 4_task_distributed.py
```

```
# необязательно переопределить хосты:
# export PGHOST_UNI=192.168.122.61
# export PGHOST_PERSONAL=192.168.122.60
```

```
# В меню: 1 / 2 / 4 / 6 / 12 / 14, выход — 0
# Для направлений и букв скрипт печатает примеры; Enter = значение по умолчанию
```

Адекватность решения. Так и задумано ТЗ части 2: «программы ... используя распределённую схему», когда личное из одной БД, университетское — из другой. Скрипт не пишет в БД (только SELECT), обрабатывает ошибку подключения, закрывает соединения в finally, не падает на пустом списке id (запрос 2).

6.3. 4_task_fdw.py — часть 3 (одно подключение + FDW)

После настройки postgres_fdw на VM11 клиенту достаточно одного connect к .61. В SQL используется JOIN на students_personal (внешняя таблица → VM2) или те же поля через VIEW. Склейку делает PostgreSQL, Python только показывает результат — это и есть отличие от части 2.

```
source .venv/bin/activate
python 4_task_fdw.py
# FDW-клиент → 192.168.122.61 (личные данные тянутся с .60 через FDW)
```

```
# export PGHOST=192.168.122.61 # при необходимости
```

Адекватность решения. Соответствует ТЗ части 3: данные студентов со второй ВМ используются «автоматически» при работе с первой. Если FDW не настроен, скрипт покажет ошибку SQL (например, нет students_personal) — это ожидаемо и помогает диагностике.

6.4. Сравнение двух клиентов

Критерий	distributed (ч.2)	fdw (ч.3)
Файл	4_task_distributed.py	4_task_fdw.py
Число connect	2 (.61 и .60)	1 (.61)
Кто склеивает ФИО	Python по id	PostgreSQL (FDW/VIEW)
Знает ли клиент IP .60	да	нет
Когда запускать	после разреза данных	после настройки FDW

Поток запросов: часть 2 vs часть 3

Один и тот же смысл лабы 1 — разный путь данных

А. Часть 2 — distributed (два хоста в клиенте)

App

```
4_task_distributed
connect .61
connect .60
join в Python
```

uni

VM11 .61

groups / grades

students без .60

rio

VM2 .60

students 600

phones

Клиент знает оба IP. Без FDW.

Б. Часть 3 — FDW (один хост в клиенте)

4_task_fdw

connect только .61

один SQL

SQL

VM11 + FDW

students $\square$ students_personal

v_students_full

SERVER $\rightarrow$.60

VM2 .60

600 читается
через FDW

Клиент не знает IP .60 — «прозрачный» доступ к внешней БД.

Рисунок U5 — потоки данных клиентов части 2 и части 3 (ещё раз)

6.5. Полный листинг 4_task_distributed.py

Ниже приведён актуальный исходный код клиента части 2 (как в каталоге сдачи / 4_task/).

Листинг — 4_task_distributed.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Задание 4 · часть 2 — распределённый клиент (две БД).

Смысл по ТЗ
-----
После разреза данных:
    • VM11 (.61) — университет (группы, направления, оценки...), ФИО обнулены;
    • VM2 (.60) — только личное (students с ФИО + phones).

Клиент лабы 1 «в лоб» на одну БД больше не работает. Этот скрипт делает то же
по смыслу, но сам ходит в ДБА PostgreSQL и склеивает строки в Python по students.id.

Запуск (на хосте Pop!_OS, обе ВМ включены, часть 2 уже выполнена)::

    cd "Выполненные задания/4_task"
    python3 -m venv .venv && source .venv/bin/activate
    pip install -r requirements.txt
    python 4_task_distributed.py

Переопределение хостов (необязательно)::

    export PGHOST_UNI=192.168.122.61
    export PGHOST_PERSONAL=192.168.122.60

Автор: Ермаков Л., М09-КИМ26
"""

from __future__ import annotations

import os
import sys
from datetime import date

import psycopg2
from psycopg2.extras import RealDictCursor # строки как dict: row["id"]
from tabulate import tabulate # красивая таблица в терминале

# --- подключения -----
# UNI = университетский контур (VM11), PERSONAL = личный (VM2).
# Пароли учебные (db1_user / 1) — как во всех лабах курса.
# os.environ.get — можно сменить IP без правки кода (удобно на защите).

VM_UNI = {
    "host": os.environ.get("PGHOST_UNI", "192.168.122.61"), # primary / university
    "port": int(os.environ.get("PGPORT_UNI", "5432")),
    "dbname": "university_db",
    "user": "db1_user",
    "password": "1",
}
VM_PERSONAL = {
    "host": os.environ.get("PGHOST_PERSONAL", "192.168.122.60"), # personal после split
    "port": int(os.environ.get("PGPORT_PERSONAL", "5432")),
    "dbname": "university_db",
    "user": "db1_user",
    "password": "1",
}

def connect(cfg: dict):
    """Открыть соединение; при ошибке — понятное сообщение и выход."""
    try:
        return psycopg2.connect(**cfg) # **cfg → host=, port=, ...
    except psycopg2.OperationalError as exc:
        # Частые причины: ВМ выключена, postgres не слушает, неверный IP/пароль
        print(f"Не удалось подключиться к {cfg.get('host')}: {exc}", file=sys.stderr)
        print("Проверьте: ВМ запущена, PostgreSQL работает, IP/пароль верны.", file=sys.stderr)
        sys.exit(1)

def show(rows, headers=None) -> None:
    """Красивая таблица в терминале (как сетка Data Editor)."""
    if not rows:
        print("\nДанные не найдены.")
        return
    # если пришли dict-строки — сами вытащим имена колонок
    if headers is None and isinstance(rows[0], dict):
        headers = list(rows[0].keys())
        rows = [[r.get(h) for h in headers] for r in rows]
    print(tabulate(rows, headers=headers, tablefmt="grid"))

def personal_map(pers, ids=None) -> dict:
    """
    Словарь id → личная карточка с VM2.
    Если ids задан — тянем только нужных студентов (меньше трафика).
    Ключ словаря = students.id — по нему потом склеиваем с university.
    """
    cur = pers.cursor(cursor_factory=RealDictCursor)
    if ids is not None:
        ids = list(ids)
        if not ids:
            return {} # пустой ANY(%) в PostgreSQL неудобен — просто выходим
        # ANY(%) — список id одним параметром (не N отдельных запросов)
        cur.execute(

```

```

        SELECT id, last_name, first_name, middle_name, birth_date, email
        FROM students WHERE id = ANY(%s)
        """
        (ids,),
    )
else:
    # без фильтра — вся personal-таблица (для возраста и т.п.)
    cur.execute(
        """
        SELECT id, last_name, first_name, middle_name, birth_date, email
        FROM students
        """
    )
return {r["id"]: r for r in cur.fetchall()}

def fio(p) -> str:
    """Собрать строку ФИО; если карточки нет — '?' (разрыв id между ВМ)."""
    if not p:
        return "?"
    return f"{p.get('last_name') or ''} {p.get('first_name') or ''} {p.get('middle_name') or ''}.strip() or
    "?"

def ask(prompt: str, default: str, examples: list[str] | None = None) -> str:
    """Ввод с подсказками: Enter оставляет значение по умолчанию."""
    print(f"\n → {prompt}")
    if examples:
        print("    примеры:", " | ".join(examples))
    print(f"    [Enter = «{default}»]")
    value = input("    ввод: ").strip()
    return value or default # пустой ввод → default

def list_directions(uni) -> list[str]:
    """Список направлений с VM11 (нужен для меню запроса 1)."""
    cur = uni.cursor()
    cur.execute("SELECT name FROM directions ORDER BY name")
    return [r[0] for r in cur.fetchall()]

# --- запросы (номера как в лабе 1) -----
# Общий приём части 2: учёба ↔ UNI, ФИО ↔ PERSONAL, склейка по id в Python.

def query_1(uni, pers) -> None:
    """Группы по направлению: учёба с .61, ФИО с .60."""
    print("\n1. Списки групп по направлению (distributed)")
    dirs = list_directions(uni)
    if dirs:
        print("    Доступные направления:")
        for i, name in enumerate(dirs, 1):
            print(f"        {i}. {name}")
        default = dirs[0] if dirs else "Информатика и вычислительная техника"
        direction = ask("Направление (скопируй строку целиком)", default, dirs[:3] or None)

    # шаг 1: с university берём group / student_id / бюджет (ФИО там NULL)
    cur = uni.cursor()
    cur.execute(
        """
        SELECT g.name, s.id, s.is_budget
        FROM students s
        JOIN groups g ON g.id = s.group_id
        JOIN directions d ON d.id = g.direction_id
        WHERE d.name = %s
        ORDER BY g.name, s.id
        """
        ,
        (direction,),
    )
    rows = cur.fetchall()

    # шаг 2: те же id → ФИО с personal (один SELECT, не по одному)
    pmap = personal_map(pers, [r[1] for r in rows])

    # шаг 3: склейка в Python (это и есть «распределённый клиент»)
    out = []
    for gname, sid, budget in rows:
        out.append(
            {
                "Группа": gname,
                "ФИО": fio(pmap.get(sid)),
                "Тип": "Бюджет" if budget else "Внебюджет",
            }
        )
    out.sort(key=lambda x: (x["Группа"], x["ФИО"]))
    show(out)

def query_2(uni, pers) -> None:
    """Сначала фильтр по фамилии на VM2, потом группы/направления с VM11."""
    print("\n2. Студенты по букве фамилии (personal → uni)")
    letter = ask("Первая буква фамилии", "И", ["А", "Е", "И", "К", "П", "С"]).upper()[1:]
    if not letter:
        print("    пустая буква — отмена")
        return

    # personal: WHERE по last_name (на .61 фамилии уже NULL — фильтр там бессмысленен)
    cur = pers.cursor(cursor_factory=RealDictCursor)
    cur.execute(
        "SELECT * FROM students WHERE UPPER(last_name) LIKE %s ORDER BY last_name",
        (f"%{letter}%",),
    )
    people = cur.fetchall()
    ids = [p["id"] for p in people]
    if not ids:

```

```

        show([])
        return

# university: дотягиваем группу и направление по тем же id
curu = uni.cursor()
curu.execute(
    """
    SELECT s.id, g.name, d.name
    FROM students s
    JOIN groups g ON g.id = s.group_id
    JOIN directions d ON d.id = g.direction_id
    WHERE s.id = ANY(%s)
    """
    ,
    (ids,),
)
uni_map = {r[0]: (r[1], r[2]) for r in curu.fetchall()}
out = []
for p in people:
    g, dname = uni_map.get(p["id"], ("?", "?")) # "?" если id нет на uni
    out.append({"ФИО": fio(p), "Группа": g, "Направление": dname})
show(out)

def query_4(uni, pers) -> None:
    """Возраст считается на клиенте из birth_date с personal-БД."""
    print("\n4. Возраст студентов (personal)")
    # uni здесь не нужен: даты рождения только на .60
    pmap = personal_map(pers)
    out = []
    today = date.today()
    for _sid, p in pmap.items():
        if not p.get("birth_date"):
            continue # пропуск, если дата пустая
        bd = p["birth_date"]
        # классический расчёт возраста: год – год, минус 1 если ДР ещё не был
        age = today.year - bd.year - ((today.month, today.day) < (bd.month, bd.day))
        out.append({"ФИО": fio(p), "Дата рождения": bd, "Возраст": age})
    out.sort(key=lambda x: x["ФИО"])
    show(out[:40]) # в терминал не вываливаем всех сразу
    print(f"показано до 40 из {len(out)}")

def query_6(uni, pers) -> None:
    """Только university: ФИО не нужны."""
    print("\n6. Количество студентов по направлениям (uni)")
    # pers не используем – счётчики живут в учебном контуре
    cur = uni.cursor()
    cur.execute(
        """
        SELECT d.name, COUNT(s.id)
        FROM directions d
        LEFT JOIN groups g ON g.direction_id = d.id
        LEFT JOIN students s ON s.group_id = g.id
        GROUP BY d.name
        ORDER BY COUNT(s.id) DESC
        """
    )
    show([{"Направление": n, "Количество": c} for n, c in cur.fetchall()])

def query_12(uni, pers) -> None:
    """Средние оценки – только university (таблица grades на .61)."""
    print("\n12. Средняя оценка по предметам (uni)")
    cur = uni.cursor()
    cur.execute(
        """
        SELECT sub.name, ROUND(AVG(gr.grade)::NUMERIC, 2)
        FROM subjects sub
        JOIN grades gr ON gr.subject_id = sub.id
        WHERE gr.grade > 2
        GROUP BY sub.name
        ORDER BY AVG(gr.grade) DESC
        """
    )
    show([{"Предмет": n, "Средняя": a} for n, a in cur.fetchall()])

def query_14(uni, pers) -> None:
    """Отличники: id с VM11, ФИО с VM2."""
    print("\n14. Отличники (distributed)")
    # HAVING MIN=5 → все оценки пятёрки (нет двоек/троек/...)
    cur = uni.cursor()
    cur.execute(
        """
        SELECT s.id, g.name
        FROM students s
        JOIN groups g ON g.id = s.group_id
        JOIN grades gr ON gr.student_id = s.id
        GROUP BY s.id, g.name
        HAVING MIN(gr.grade) = 5 AND COUNT(gr.id) > 0
        """
    )
    rows = cur.fetchall()
    pmap = personal_map(pers, [r[0] for r in rows]) # ФИО только для отличников
    show([{"ФИО": fio(pmap.get(i)), "Группа": g} for i, g in rows])

# ключ меню → (подпись, функция); номера совпадают с лабой 1
MENU = {
    "1": ("Списки групп по направлению (distributed)", query_1),
    "2": ("Студенты по букве фамилии (personal-uni)", query_2),
    "4": ("Возраст (personal)", query_4),
    "6": ("Кол-во по направлениям (uni)", query_6),
    "12": ("Средняя оценка (uni)", query_12),
    "14": ("Отличники (distributed)", query_14),
}

```

```

    "0": ("Выход", None),
}

def main() -> None:
    # два независимых TCP-соединения — суть части 2
    uni = connect(VM_UNI)
    pers = connect(VM_PERSONAL)
    print("Подключено: UNI=", VM_UNI["host"], " PERSONAL=", VM_PERSONAL["host"])
    try:
        while True:
            print("\n" + "=" * 60)
            print(" ЗАДАНИЕ 4 · РАСПРЕДЕЛЁННЫЙ КЛИЕНТ (часть 2)")
            for k, (title, _) in MENU.items():
                print(f"  [{k}] {title}")
            print("  примеры выбора: 1 | 2 | 14 | 0")
            choice = input("Выбор: ").strip()
            if choice == "0":
                break
            if choice in MENU and MENU[choice][1]:
                try:
                    MENU[choice][1](uni, pers) # вызываем выбранный query_*
                except psycopg2.Error as exc:
                    # SQL-ошибка не роняет весь клиент — можно выбрать другой пункт
                    print(f"Ошибка SQL: {exc}", file=sys.stderr)
                    input("Enter — вернуться в меню...")
            else:
                print("  нет такого пункта, примеры: 1, 2, 4, 6, 12, 14, 0")
    finally:
        # всегда закрываем оба соединения (даже при Ctrl+C / ошибке)
        uni.close()
        pers.close()

if __name__ == "__main__":
    main()

```

6.6. Полный листинг 4_task_fdw.py

Ниже приведён актуальный исходный код клиента части 3.

Листинг — 4_task_fdw.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Задание 4 · часть 3 — FDW-клиент (одно подключение).

Смысл по ТЗ
-----
На VM11 уже настроен postgres_fdw:
    SERVER personal_server → 192.168.122.60
    FOREIGN TABLE students_personal / phones_personal
    VIEW v_students_full = students [] students_personal

Клиент подключается ТОЛЬКО к VM11 (.61). ФИО PostgreSQL сам читает с VM2
через FDW — второго connect в Python нет (в отличие от части 2).

Запуск (на хосте, часть 3 уже настроена)::

    cd "Выполненные задания/4_task"
    source .venv/bin/activate
    python 4_task_fdw.py

Хост по умолчанию 192.168.122.61, можно переопределить::

    export PGHOST=192.168.122.61

Автор: Ермаков Л., М09-КИИ26
"""

from __future__ import annotations

import os
import sys

import psycopg2
from tabulate import tabulate

# одно подключение: только VM11; .60 «прячется» за FDW на сервере
DB = {
    "host": os.environ.get("PGHOST", "192.168.122.61"),
    "port": int(os.environ.get("PGPORT", "5432")),
    "dbname": "university_db",
    "user": "db1_user",
    "password": "1",
}

def connect():
    """Connect к .61; если VM/FDW не готовы — сразу понятная ошибка."""
    try:
        return psycopg2.connect(**DB)
    except psycopg2.OperationalError as exc:
        print(f"Не удалось подключиться к {DB['host']}: {exc}", file=sys.stderr)
        print("Проверьте VM11, PostgreSQL и что FDW настроен (часть 3).", file=sys.stderr)
        sys.exit(1)

def show(cur) -> None:
    """Печать результата текущего курсора (заголовки из description)."""
    rows = cur.fetchall()
    if not rows:
        print("нуто")
        return
    # cur.description[i][0] — имя колонки из SELECT ... AS "...
    print(tabulate(rows, headers=[d[0] for d in cur.description], tablefmt="grid"))

def ask(prompt: str, default: str, examples: list[str] | None = None) -> str:
    """Интерактивный ввод; Enter → default."""
    print(f"\n → {prompt}")
    if examples:
        print("    примеры:", " | ".join(examples))
    print(f"    [Enter = «{default}»]")
    value = input("    ввод: ").strip()
    return value or default

def list_directions(conn) -> list[str]:
    """Локальная таблица directions на .61 (FDW тут не нужен)."""
    with conn.cursor() as cur:
        cur.execute("SELECT name FROM directions ORDER BY name")
        return [r[0] for r in cur.fetchall()]

# пункты меню = номера запросов лабы 1 (где нужны ФИО — через FDW)
QUERIES = {
    "1": "Группы по направлению (FDW)",
    "2": "Фамилия на букву (FDW)",
    "4": "Возраст (FDW)",
    "6": "Кол-во по направлениям",
    "12": "Средняя оценка",
    "14": "Отличники (FDW ФИО)",
}

def run_query(conn, ch: str) -> None:
    """
    Все запросы идут в один хост (.61).
    """
```

Где нужны ФИО – JOIN на students_personal (внешняя таблица – VM2).
Склейку делает PostgreSQL, не Python (отличие от 4_task_distributed.py).

```
"""
title = QUERIES[ch]
print("\n", title)

if ch == "1":
    # local students + remote ФИО через FOREIGN TABLE
    dirs = list_directions(conn)
    if dirs:
        print(" Доступные направления:")
        for i, name in enumerate(dirs, 1):
            print(f" {i}. {name}")
    default = dirs[0] if dirs else "Информатика и вычислительная техника"
    direction = ask("Направление (скопируй строку целиком)", default, dirs[:3] or None)
    sql = """
SELECT g.name AS "Группа",
       sp.last_name || ' ' || sp.first_name || ' '
       || COALESCE(sp.middle_name, '') AS "ФИО",
       CASE WHEN s.is_budget THEN 'Бюджет' ELSE 'Внебюджет' END AS "Тип"
FROM students s
JOIN students_personal sp ON sp.id = s.id -- FDW → VM2
JOIN groups g ON g.id = s.group_id
JOIN directions d ON d.id = g.direction_id
WHERE d.name = %s
ORDER BY g.name, sp.last_name
"""
    params = tuple | None = (direction,)
elif ch == "2":
    # фильтр по фамилии сразу на внешней таблице (данные с .60)
    letter = ask("Первая буква фамилии", "И", ["А", "Е", "И", "К", "П", "С"]).upper()[1:]
    if not letter:
        print(" пустая буква – отмена")
        return
    sql = """
SELECT sp.last_name || ' ' || sp.first_name AS "ФИО",
       g.name AS "Группа", d.name AS "Направление"
FROM students_personal sp
JOIN students s ON s.id = sp.id
JOIN groups g ON g.id = s.group_id
JOIN directions d ON d.id = g.direction_id
WHERE UPPER(sp.last_name) LIKE %s
ORDER BY sp.last_name
"""
    params = (f'{letter}%',)
elif ch == "4":
    # возраст считает уже PostgreSQL (AGE), birth_date читается с VM2 через FDW
    sql = """
SELECT sp.last_name || ' ' || sp.first_name AS "ФИО",
       sp.birth_date,
       EXTRACT(YEAR FROM AGE(CURRENT_DATE, sp.birth_date))::INT AS "Возраст"
FROM students_personal sp
ORDER BY sp.last_name
LIMIT 30
"""
    params = None
elif ch == "6":
    # только локальные таблицы – FDW не участвует
    sql = """
SELECT d.name, COUNT(s.id)
FROM directions d
LEFT JOIN groups g ON g.direction_id = d.id
LEFT JOIN students s ON s.group_id = g.id
GROUP BY d.name ORDER BY 2 DESC
"""
    params = None
elif ch == "12":
    # grades / subjects локально на university (.61)
    sql = """
SELECT sub.name, ROUND(AVG(gr.grade)::NUMERIC, 2)
FROM subjects sub
JOIN grades gr ON gr.subject_id = sub.id
WHERE gr.grade > 2
GROUP BY sub.name
ORDER BY 2 DESC
"""
    params = None
elif ch == "14":
    # отличники локально по grades, ФИО – через students_personal
    sql = """
SELECT sp.last_name || ' ' || sp.first_name AS "ФИО", g.name
FROM students s
JOIN students_personal sp ON sp.id = s.id
JOIN groups g ON g.id = s.group_id
JOIN grades gr ON gr.student_id = s.id
GROUP BY s.id, sp.last_name, sp.first_name, g.name
HAVING MIN(gr.grade) = 5 AND COUNT(gr.id) > 0
ORDER BY sp.last_name
"""
    params = None
else:
    return

with conn.cursor() as cur:
    # params=None → запрос без плейсхолдеров %s
    cur.execute(sql, params)
    show(cur)

def main() -> None:
    conn = connect()
    print("FDW-клиент →", DB["host"], "(личные данные тянутся с .60 через FDW)")
    try:
        while True:
            print("\n" + "=" * 50)
```

```

print(" ЗАДАНИЕ 4 : FDW-КЛИЕНТ (часть 3, одно подключение)")
for k, t in QUERIES.items():
    print(f"    [{k}] {t}")
print("    [0] Выход")
print("    примеры выбора: 1 | 2 | 14 | 0")
ch = input("Выбор: ").strip()
if ch == "0":
    break
if ch not in QUERIES:
    print("    нет такого пункта, примеры: 1, 2, 4, 6, 12, 14, 0")
    continue
try:
    run_query(conn, ch)
except psycopg2.Error as exc:
    # типично: нет FOREIGN TABLE / SERVER / user mapping
    print(f"Ошибка SQL (проверьте FDW / students_personal): {exc}", file=sys.stderr)
    input("Enter — вернуться в меню...")
finally:
    conn.close() # одно соединение — закрываем в finally

if __name__ == "__main__":
    main()

```

7. Дополнительные скрины из журнала выполнения

Ниже — оставшиеся кадры из файла «как я выполнял 4 задание.pdf» (все вложенные изображения img-000...img-023), чтобы в отчёте был полный визуальный след работы.

```
db1@db-server-11:~$ sudo cat /etc/postgresql/16/main/conf.d/99-replication.conf
listen_addresses = '*'
wal_level = replica
max_wal_senders = 10
max_replication_slots = 10
hot_standby = on
wal_keep_size = 256MB
db1@db-server-11:~$ echo 1 | sudo -S bash -c 'grep -q "lab4 replication" /etc/postgresql/16/main/pg_hba.conf || cat >> /etc/postgresql/16/main/pg_hba.conf <<EOF'

# lab4 replication / app
host      replication      replicator      192.168.122.0/24      scram-sha-256
host      all              all             192.168.122.0/24      scram-sha-256
EOF'
```

Рисунок Г.0 — Создание/проверка replication slot на VM11

```
db1@db-server-11:~$ echo 1 | sudo -S bash -c 'grep -q "lab4 replication" /etc/postgresql/16/main/pg_hba.conf || cat >> /etc/postgresql/16/main/pg_hba.conf <<EOF'

# lab4 replication / app
host      replication      replicator      192.168.122.0/24      scram-sha-256
host      all              all             192.168.122.0/24      scram-sha-256
EOF'
db1@db-server-11:~$ echo 1 | sudo -S -u postgres psql <<'SQL'
DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'replicator') THEN
        CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD '1';
    ELSE
        ALTER ROLE replicator WITH REPLICATION LOGIN PASSWORD '1';
    END IF;
END$$;

SELECT pg_create_physical_replication_slot('slot_lab4')
WHERE NOT EXISTS (
    SELECT 1 FROM pg_replication_slots WHERE slot_name = 'slot_lab4'
);
SQL
DO
    pg_create_physical_replication_slot
-----
(slot_lab4,)
(1 row)
```

Рисунок Г.1 — pg_hba + SQL роли replicator / slot_lab4

```
db1@db-server-11:~$ echo 1 | sudo -S -u postgres psql <<'SQL'
DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'replicator') THEN
        CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD '1';
    ELSE
        ALTER ROLE replicator WITH REPLICATION LOGIN PASSWORD '1';
    END IF;
END$$;

SELECT pg_create_physical_replication_slot('slot_lab4')
WHERE NOT EXISTS (
    SELECT 1 FROM pg_replication_slots WHERE slot_name = 'slot_lab4'
);
SQL
DO
    pg_create_physical_replication_slot
-----
(slot_lab4,)
(1 row)

db1@db-server-11:~$ echo 1 | sudo -S systemctl restart postgresql
sleep 2
echo 1 | sudo -S -u postgres psql -c "SHOW wal_level;"
echo 1 | sudo -S -u postgres psql -c "SELECT slot_name, active FROM pg_replication_slots;"
wal_level
-----
replica
(1 row)

slot_name | active
-----+-----
slot_lab4 | f
(1 row)
```

Рисунок Г.2 — Очистка PGDATA на VM2 перед basebackup

```
db1@db-server-2:~$ sudo -u postgres PGPASSWORD=1 pg_basebackup \
-h 192.168.122.61 \
-p 5432 \
-U replicator \
-D /var/lib/postgresql/16/main \
-Fp -Xs -P -R \
-S slot_lab4
31612/31612 kB (100%), 1/1 tablespace
db1@db-server-2:~$
```

Рисунок Г.3 — Успешный pg_basebackup с .61

```
db1@db-server-2:~$ sudo chown -R postgres:postgres /var/lib/postgresql/16/main
sudo chmod 700 /var/lib/postgresql/16/main
sudo systemctl start postgresql
sleep 2
sudo -u postgres psql -c "SELECT pg_is_in_recovery();"
sudo -u postgres psql -d university_db -c "SELECT count(*) FROM students;"
pg_is_in_recovery
-----
t
(1 row)

count
-----
240
(1 row)
```

Рисунок Г.4 — Старт standby и проверка recovery/count

```
db1@db-server-11:~$ sudo -u postgres psql -c "SELECT slot_name, active FROM pg_replication_slots;"
sudo -u postgres psql -c "SELECT client_addr, state, sync_state FROM pg_stat_replication;"
sudo -u postgres psql -d university_db -c "SELECT count(*) FROM students;"
[sudo] password for db1:
slot_name | active
-----+-----
slot_lab4 | t
(1 row)

client_addr | state | sync_state
-----+-----+-----
192.168.122.60 | streaming | async
(1 row)

count
-----
240
(1 row)
```

Рисунок Г.5 — Проверка streaming на primary

ВМ	IP	После части 2
VM11	.61	Университет: группы, предметы, оценки, посещаемость... без ФИО/ телефонов (или ФИО обнулены)
VM2	.60	Только личное: студенты (ФИО, адрес...) + телефоны

Рисунок Г.6 — Доп. фрагмент проверки части 1

Чем отличается от части 1

	Часть 1	Часть 2
Данные	Одинаковые на обеих	Разные: учёба / личное
Связь VM	Streaming replication	Два независимых сервера
Клиент	Один хост (как лаба 1)	Два PGHOST / два соединения

Рисунок Г.7 — Сравнение часть 1 vs часть 2

```
serafim@pop-os:~$ virsh --connect qemu:///system snapshot-list ubuntu24.04-11
Name                Creation Time          State
-----
before-lab4         2026-09-17 20:27:24 +0300  running
```

Рисунок Г.8 — Список снимков before-lab4

```
Domain snapshot before-lab4 reverted

serafim@pop-os:~$ virsh --connect qemu:///system start ubuntu24.04-11
sleep 15
ssh db1@192.168.122.61
error: Domain is already active

db1@192.168.122.61's password:
Welcome to Ubuntu 24.04.5 LTS (GNU/Linux 6.8.0-139-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Thu Sep 17 04:41:07 PM UTC 2026

System load:  1.44      Processes:           201
Usage of /:   56.3% of 24.44GB   Users logged in:    0
Memory usage: 67%      IPv4 address for enp1s0: 192.168.122.61
Swap usage:   12%

 * Canonical Workshop gives developers fast, composable, reproducible, and
   secure developer environments that are perfect for agentic workflows.
   https://ubuntu.com/workshop

Expanded Security Maintenance for Applications is not enabled.

1 update can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check
your Internet connection or proxy settings

Last login: Thu Sep 17 16:41:08 2026 from 192.168.122.1
db1@db-server-11:~$
```

Рисунок Г.9 — Откат / подготовка части 2

```
db1@db-server-11:~$ hostname
db-server-11
[sudo] password for db1:
pg_is_in_recovery
-----
f
(1 row)

count
-----
240
(1 row)

with_fio
-----
240
(1 row)

db1@db-server-11:~$
```

Рисунок Г.10 — VM11 после отката к снимку

```
db1@db-server-2:~$ sudo -u postgres psql -c "SELECT pg_promote();"
[sudo] password for db1:
pg_promote
-----
t
(1 row)

db1@db-server-2:~$
```

Рисунок Г.11 — Promote на VM2

```
db1@db-server-2:~$ sudo -u postgres psql -c "SELECT pg_is_in_recovery();"
sudo -u postgres psql -d university_db -c "
SELECT count(*) AS students,
       count(*) FILTER (WHERE last_name IS NOT NULL) AS with_fio
FROM students;"
pg_is_in_recovery
-----
f
(1 row)

students | with_fio
-----+-----
240 | 240
(1 row)
```

Рисунок Г.12 — Проверка после promote

```
SQL> ALL SELECT 'phones', count(*) FROM phones;
BEGIN
NOTICE: table "_personal_students" does not exist, skipping
DROP TABLE
SELECT 240
NOTICE: table "_personal_phones" does not exist, skipping
DROP TABLE
SELECT 320
DROP TABLE
DROP TABLE
DROP TABLE
DROP TABLE
DROP TABLE
DROP TABLE
DROP TABLE
DROP TABLE
DROP TABLE
CREATE TABLE
INSERT 0 240
CREATE TABLE
INSERT 0 320
DROP TABLE
DROP TABLE
GRANT
GRANT
COMMIT

List of relations
Schema | Name | Type | Owner
-----+-----+-----+-----
public | phones | table | postgres
public | students | table | postgres
(2 rows)

t | count
--+-----
students | 240
phones | 320
(2 rows)
```

Рисунок Г.13 — Разрез personal на VM2 (SQL/результат)

```

count(*) FILTER (WHERE last_name IS NULL) AS anon
FROM students;
SELECT count(*) FROM grades;
SELECT count(*) FROM groups;
SQL
[sudo] password for db1:
BEGIN
DROP TABLE
ALTER TABLE
UPDATE 240
COMMIT

```

Schema	Name	Type	Owner
public	attendance	table	postgres
public	directions	table	postgres
public	grades	table	postgres
public	groups	table	postgres
public	students	table	postgres
public	subjects	table	postgres
public	teacher_subjects	table	postgres
public	teachers	table	postgres
public	time_slots	table	postgres

(9 rows)

```

students | anon
-----+-----
      240 |   240
(1 row)

count
-----
    678
(1 row)

count
-----
     15
(1 row)

```

Рисунок Г.14 — Продолжение разреза / проверка VM2

```

Connection to 192.168.122.61 closed.
serafim@pop-os:~$ cd "/home/serafim/Рабочий стол/РБД/Выполненные задания/4_task"
source .venv/bin/activate
python 4_task_distributed.py
✓ Подключено: UNI= 192.168.122.61  PERSONAL= 192.168.122.60

=====
ЗАДАНИЕ 4 · РАСПРЕДЕЛЁННЫЙ КЛИЕНТ
[1] Списки групп по направлению (distributed)
[2] Студенты по букве фамилии (personal-uni)
[4] Возраст (personal)
[6] Кол-во по направлениям (uni)
[12] Средняя оценка (uni)
[14] Отличники (distributed)
[0] Выход
Выбор: 1

```

Рисунок Г.15 — Разрез university на VM11

```
1. Списки групп по направлению
Доступные направления:
1. Информатика и вычислительная техника
2. Информационная безопасность
3. Прикладная математика и информатика
4. Программная инженерия
5. Системный анализ и управление

- Направление (скопируй строку целиком)
примеры: Информатика и вычислительная техника | Информационна
я безопасность | Прикладная математика и информатика
[Enter = «Информатика и вычислительная техника»]
ввод: Информатика и вычислительная техника

+-----+-----+-----+
| Группа | ФИО | Тип |
+-----+-----+-----+
| ИВТ-101 | Богданов Роман Денисович | Бюджет |
+-----+-----+-----+
| ИВТ-101 | Виноградов Артём Игоревич | Бюджет |
+-----+-----+-----+
| ИВТ-101 | Волков Андрей Сергеевич | Внебюджет |
+-----+-----+-----+
| ИВТ-101 | Воробьев Максим Александрович | Внебюджет |
+-----+-----+-----+
| ИВТ-101 | Голубев Сергей Павлович | Бюджет |
+-----+-----+-----+
| ИВТ-101 | Зайцев Михаил Олегович | Бюджет |
+-----+-----+-----+
| ИВТ-101 | Иванов Иван Иванович | Бюджет |
+-----+-----+-----+
| ИВТ-101 | Кузнецов Алексей Дмитриевич | Бюджет |
+-----+-----+-----+
| ИВТ-101 | Михайлов Тимофей Андреевич | Бюджет |
+-----+-----+-----+
| ИВТ-101 | Павлов Николай Викторович | Бюджет |
+-----+-----+-----+
| ИВТ-101 | Петров Петр Петрович | Бюджет |
```

Рисунок Г.16 — Distributed-клиент (часть 2)

Что получится	
Объект на VM11	Смысл
SERVER personal_server	Ссылка на VM2 (.60)
students_personal	Внешняя таблица = students на VM2
phones_personal	Внешняя таблица = phones на VM2
v_students_full	View: локальный студент + ФИО с VM2

Рисунок Г.17 — Схема объектов FDW

	Часть 2	Часть 3
Кто склеивает	Python	PostgreSQL (FDW + view)
Куда подключается клиент	два хоста	в основном один (.61)
Где лежат ФИО физически	только VM2	всё ещё только VM2, но видны «как свои» на VM11

Рисунок Г.18 — Сравнение часть 2 vs часть 3

```

db1@db-server-11:~$ sudo -u postgres psql -d university_db <<'SQL'
CREATE EXTENSION IF NOT EXISTS postgres_fdw;

DROP USER MAPPING IF EXISTS FOR db1_user SERVER personal_server;
DROP USER MAPPING IF EXISTS FOR postgres SERVER personal_server;
DROP SERVER IF EXISTS personal_server CASCADE;

-- «МОСТ» к VM2
CREATE SERVER personal_server
  FOREIGN DATA WRAPPER postgres_fdw
  OPTIONS (host '192.168.122.60', port '5432', dbname 'university_db');

-- под каким логином ходим на VM2
CREATE USER MAPPING FOR db1_user
  SERVER personal_server
  OPTIONS (user 'db1_user', password '1');

CREATE USER MAPPING FOR postgres
  SERVER personal_server
  OPTIONS (user 'db1_user', password '1');
SQL
CREATE EXTENSION
NOTICE: server "personal_server" does not exist, skipping
DROP USER MAPPING
NOTICE: server "personal_server" does not exist, skipping
DROP USER MAPPING
NOTICE: server "personal_server" does not exist, skipping
DROP SERVER
CREATE SERVER
CREATE USER MAPPING
CREATE USER MAPPING

```

Рисунок Г.19 — Настройка FDW (терминал)

```

        middle_name VARCHAR(100),
        birth_date DATE,
        city VARCHAR(100),
        street VARCHAR(100),
        house_number VARCHAR(20),
        email VARCHAR(100)
    ) SERVER personal_server
    OPTIONS (schema_name 'public', table_name 'students');

CREATE FOREIGN TABLE phones_personal (
    id INTEGER,
    student_id INTEGER,
    phone_number VARCHAR(20)
) SERVER personal_server
OPTIONS (schema_name 'public', table_name 'phones');

CREATE OR REPLACE VIEW v_students_full AS
SELECT
    s.id,
    sp.last_name,
    sp.first_name,
    sp.middle_name,
    sp.birth_date,
    sp.city,
    sp.street,
    sp.house_number,
    sp.email,
    s.group_id,
    s.is_budget
FROM students s
JOIN students_personal sp ON sp.id = s.id;

GRANT USAGE ON FOREIGN SERVER personal_server TO db1_user;
SQL> SELECT ON v_students_full TO db1_user::;
NOTICE: foreign table "students_personal" does not exist, skipping
DROP FOREIGN TABLE
NOTICE: foreign table "phones_personal" does not exist, skipping
DROP FOREIGN TABLE
CREATE FOREIGN TABLE
CREATE FOREIGN TABLE
CREATE VIEW
GRANT
GRANT
GRANT
GRANT

```

Рисунок Г.20 — CREATE FOREIGN TABLE / VIEW

```
db1@db-server-11:~$ sudo -u postgres psql -d university_db -c "
SELECT count(*) FROM students_personal;
SELECT count(*) FROM v_students_full;
SELECT id, last_name, first_name FROM v_students_full ORDER BY id LIMIT 5;
"

count
-----
      240
(1 row)

count
-----
      240
(1 row)

 id | last_name | first_name
-----+-----
  1 | Иванов   | Иван
  2 | Петров   | Петр
  3 | Сидоров  | Сидор
  4 | Кузнецов | Алексей
  5 | Смирнов  | Дмитрий
(5 rows)
```

Рисунок Г.21 — Проверка v_students_full

```
sk/4_task_distributed.py", line 245, in main
    choice = input("Выбор: ").strip()
KeyboardInterrupt

(.venv) serafim@pop-os:~/Рабочий стол/РБД/Выполненные задания/4
_task$ cd "/home/serafim/Рабочий стол/РБД/Выполненные задания/4
_task"
source .venv/bin/activate
python 4_task_fdw.py
✓ FDW-клиент → 192.168.122.61 (личные данные тянутся с .60 че
рез FDW)

=====
ЗАДАНИЕ 4 · FDW-КЛИЕНТ (одно подключение)
[1] Группы по направлению (FDW)
[2] Фамилия на букву (FDW)
[4] Возраст (FDW)
[6] Кол-во по направлениям
[12] Средняя оценка
[14] Отличники (FDW ФИО)
[0] Выход
примеры выбора: 1 | 2 | 14 | 0
Выбор: 
```

Рисунок Г.22 — FDW-клиент Python

Итог задания 4		
Часть	Суть	Статус
1	Снимок + зеркало VM11→VM2	✓
2	Разрез + Python на две БД	✓
3	FDW на VM11 + клиент на .61	✓

Для сдачи сохрани скриншоты всех трёх частей. Если нужно — обновим PDF-отчёт под VM11

Рисунок Г.23 — Итоговая таблица статуса частей

8. Итог выполнения

Часть	Что сделано	Доказательство
1	Снимок + streaming VM11→VM2	slot active, streaming .60, recovery=t, COUNT=240
2	Разрез + Python на 2 БД	VM2: students/phones; VM11: anon; 4_task_distributed.py
3	FDW + клиент на .61	students_personal, v_students_full, 4_task_fdw.py

Источники скринов: «4 задание/как я выполнял 4 задание.pdf», «4 задание/Скрины таблиц VM11», рабочие терминальные проверки. Клиенты: 4_task_distributed.py, 4_task_fdw.py. Пересборка отчёта: python build_report.py.